Desenvolvimento Web

Pablo Leon Rodrigues

UPF

1. Desenvolvimento Web
    1. Breve resumo da história da Web
    2. Como funciona?
    3. URI
    4. DNS Lookup
    5. TCP
    6. HTTP
    7. Request & Response
    8. Status Code
2. HTML
    1. *Parsing* HTML
    2. *DOM*
    3. *Parsing* CSS
    4. Javascript
    5. Paint
    6. Referências

Breve resumo da história da Web

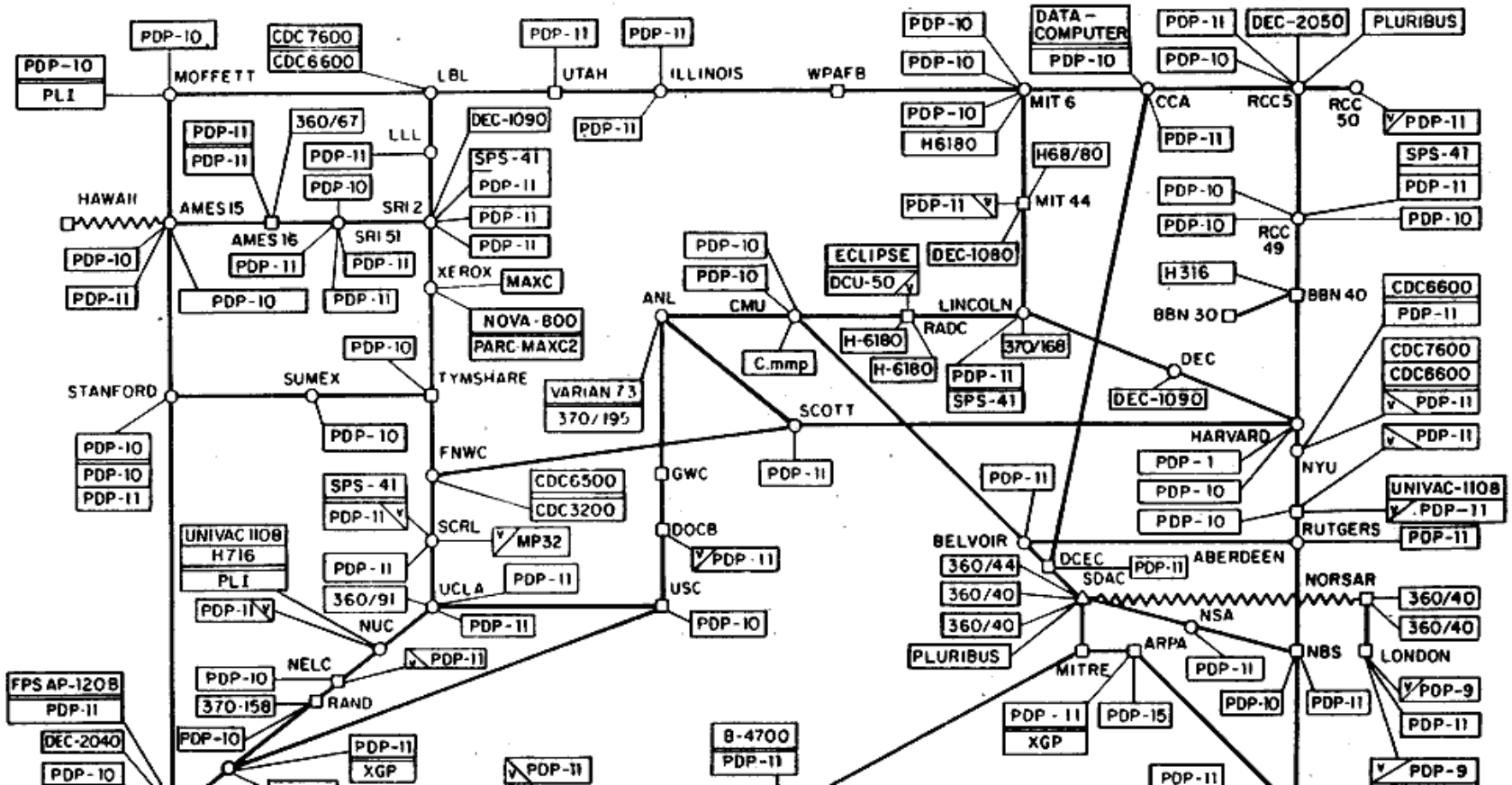
A world wide web(www) como conheçemos hoje é produto de vários trabalhos e pesquisas, mas principalmente reconhecida como trabalho de Sir Tim Berners-Lee, desenvolvido no CERN em 1989.

Ele é creditado como criador das especificações URIs, HTTP and HTML.

Camadas da Web

- **Internet** ARPANET 1969, *J.C.R Licklider* e TCP/IP 1975, *Cerf & Kahn*
- **Email** 1971, *Ray Tomlinson*
- **Browsers** DOS Houdini 1986, *Neil Larson* e Mosaic 1993, *Marc Andreessen*

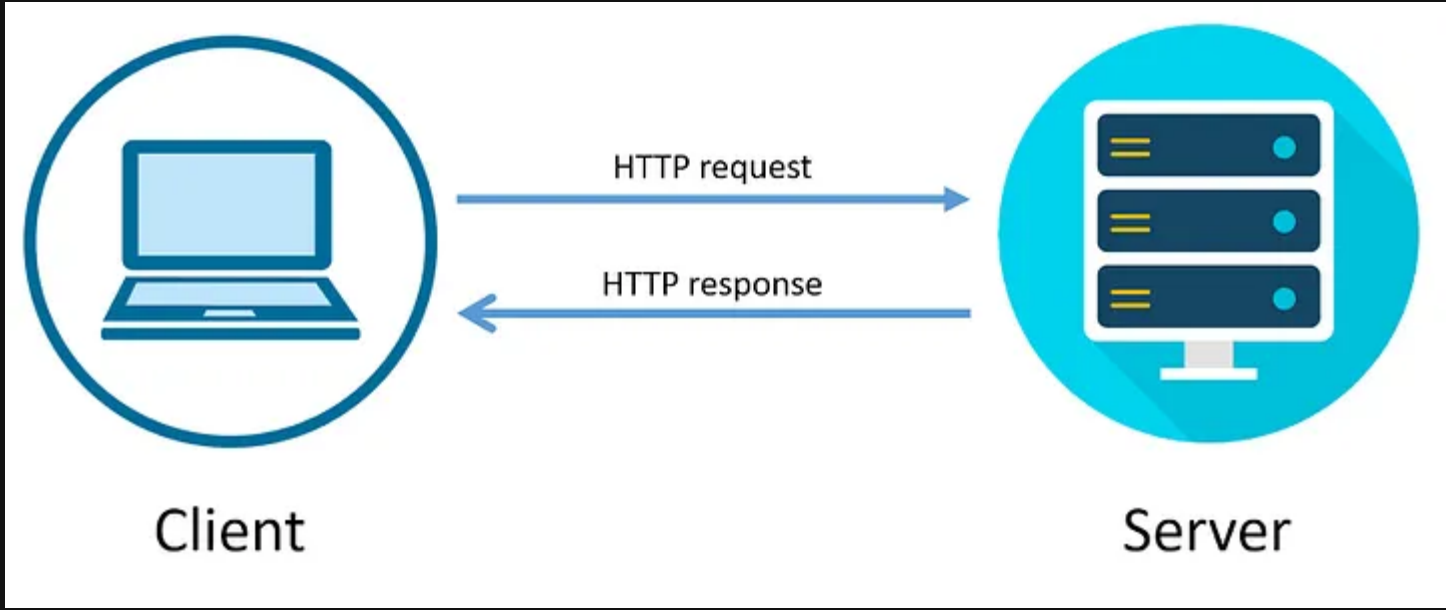
ARPANET LOGICAL MAP, MARCH 1977



Como funciona?

Computadores(no sentido amplo da palavra) conectados a internet assumem o papel de cliente ou servidor.

O cliente faz uma requisição para o servidor, que responde com algum conteúdo.



URI

URI(*Uniform Resource Identifier*) é uma sequência de caracteres utilizada para identificar um recurso na rede. O tipo mais comum de URI é a URL(*Uniform Resource Locator*)

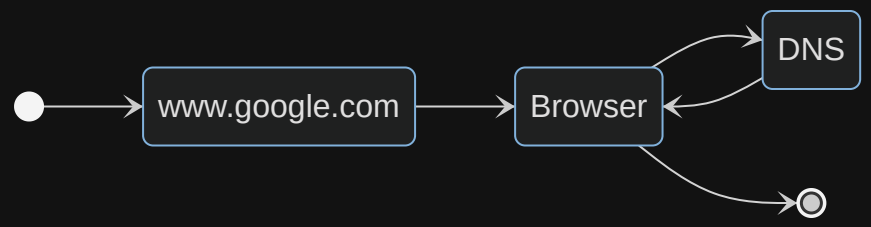
Anatomia da URL

```
          userinfo       host      port
          ┌──────┐  ┌────────────┐ ┌┐
https://john.doe@www.example.com:123/forum/?tag=networking&order=newest#top
└───┘    └──────────────────────────┘└──┘ └──────────────────────────┘ └┘
scheme          authority            path              query         fragment

mailto:John.Doe@example.com
└────┘ └──────────────────┘
scheme     path
```

DNS Lookup

- Para acessar um determinado site ou aplicativo você digita uma URL (exemplo, www.google.com) na barra de endereço do navegador.
- Essa URL é um nome feito para ser lido por humanos, a comunicação entre cliente e servidor é feita utilizado endereços numéricos chamados endereços de IP(*IP address*).
- Para fazer essa conversão de URL para IP, o *browser* se comunica com um servidor DNS(*Domain Name System*). Pense nele como uma agenda telefonica. Nome > número
- O DNS procura o endereço associado a URL enviada e retorna esse IP para o *browser*



TCP

TCP(*Transmission Control Protocol*, Protocolo de Controle de Transmissão) é um protocolo de comunicação que permite que programas aplicativos e dispositivos de computação troquem mensagens em uma rede. Ele foi projetado para enviar pacotes (de dados) pela Internet e garantir a entrega bem-sucedida de dados e mensagens pela rede.

IP

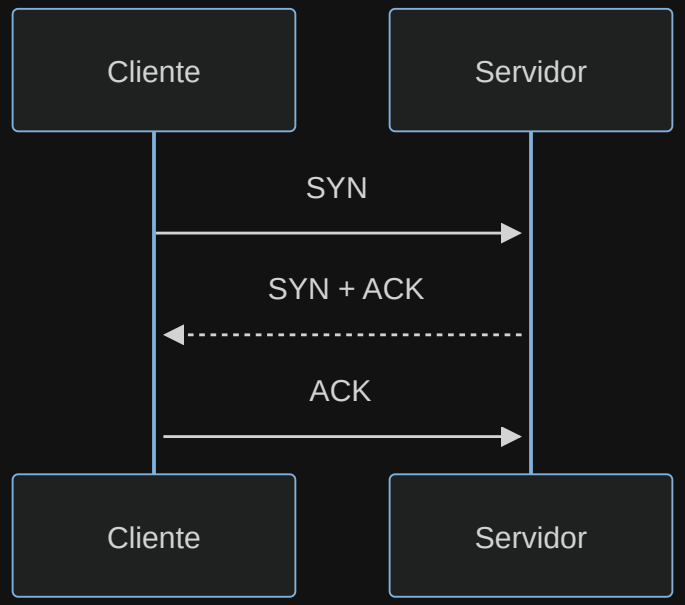
IP(*Internet Protocol*, Protocolo de internet) é responsável por fazer o roteamento dos pacotes de dados pela Internet com base em seus endereços IP.

Handshake

- Com o endereço IP em mãos, o navegador tenta iniciar uma conexão TCP com o servidor web nesse endereço.
  - Também conhecida como *Three-way handshake* ou SYN-SYN-ACK o *TCP Handshake* é a comunicação necessária para estabelecer a sessão TCP.

O TCP Handshake é um mecanismo projetado para que duas entidades (no nosso caso, o cliente e o servidor) que desejam passar informações entre si possam negociar os parâmetros da conexão antes de transmitir os dados.

- O navegador envia uma mensagem `SYNC` para o servidor solicitando SYNchronization (*synchronization means the connection*)
- O servidor responde com uma mensagem `SYN + ACK` (*SYNChronization and ACKnowledgement*)
- O cliente então finaliza enviando uma mensagem `ACK` (*ACKnowledgement*)



HTTP

O *Hypertext Transfer Protocol*(HTTP) é um protocolo que define como é feita a comunicação entre cliente e servidor. Conceitos-chave:

- *Methods* : Definem o que o servidor deve fazer com o request, sendo:
  - GET para recuperar informação
  - POST/PUT para enviar dados
  - DELETE remover dados
- URL: caminho do site ou aplicativo desejado
- *Headers* : Fornecem informação adicional sobre os dados sendo enviados, preferências de linguagem, cookies

Request & Response

General

```
Request URL: https://leon.dev.br/
Request Method: GET
Status Code: 200 OK
Remote Address: 185.199.108.153:443
Referrer Policy: strict-origin-when-cross-origin
```

Request Headers

```
Sec-Ch-Ua: "Not A(Brand";v="99", "Google Chrome";
    v="121", "Chromium";v="121"
Sec-Ch-Ua-Mobile: ?1
Sec-Ch-Ua-Platform: "Android"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Linux; Android 6.0;
    Nexus 5 Build/MRA58N) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/121.0.0.0 Mobile
    Safari/537.36
```

Response Headers

```
Accept-Ranges: bytes
Access-Control-Allow-Origin: *
Age: 0
Cache-Control: max-age=600
Content-Encoding: gzip
Content-Length: 3553
Content-Type: text/html; charset=utf-8
Date: Tue, 13 Feb 2024 22:30:23 GMT
Expires: Tue, 13 Feb 2024 22:40:23 GMT
Last-Modified: Sun, 04 Feb 2024 13:52:33 GMT
Server: GitHub.com
Vary: Accept-Encoding
X-Fastly-Request-Id:
    9faddf5d2c8006c085438f7c68a7cbf80a3d4fe7
X-Github-Request-Id:
    CF3A:65E93:116AA8:142208:65CB93E8
X-Served-By: cache-gru-sbgr1930075-GRU
X-Timer: S1707863423.250750,VS0,VE143
```

Status Code

Depois de processar o que foi solicitado no request ele envia o response de volta para o browser contendo:

- *Status Code*: indica se a solicitação foi bem-sucedida (por exemplo, 200 para OK, 404 para Não encontrado).
- *Headers*: fornecem informações adicionais sobre a resposta, como o tipo de conteúdo (por exemplo, HTML, imagem).
- *Body*: Contém os dados reais solicitados, normalmente em formato HTML para páginas da web.

O Status Code fornece informações sobre o resultado do request. Os status são agrupados em:

- 1xx - Informativo
- 2xx - Sucesso
- 3xx - Redirecionamento
- 4xx - Erro no cliente
- 5xx - Erro no servidor

200 - Retorno com sucesso em um HTTP request

404 - O *resource* solicitado não foi encontrado

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello World!</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="hello-message">Hello World!</h1>
  <script src="script.js"></script>
</body>
</html>
```

Parsing HTML

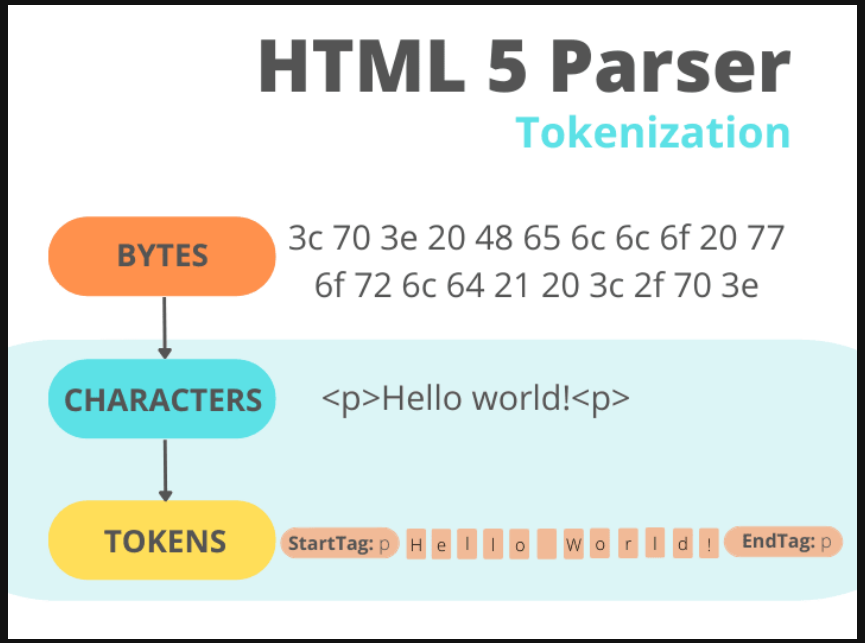
Após receber o conteúdo do *response*, o navegador vai começar a fazer o *Parsing* dos dados. **Parsing** significa analisar e converter um programa em um formato interno para ser executado.

O Navegador vai então usar um componente chamado *Engine* ou motor para fazer o *Parsing*. Existem diversos tipos de *Engine*:

- Blink(Chromium): Chrome, Edge, Brave, Vivaldi, Opera... Escrito em `C++` e mantido pelo Google.
- Gecko: Firefox. Escrito em `C++ e JavaScript`, recentemente começou ter partes escritas em `Rust` e mantido pela Mozilla.
- WebKit: Safari, Otter, Epiphany, escrito em `C++` e mantido pela Apple.

Tokenization

No processo de *Tokenization*, é feita uma análise léxica do documento e convertido em tokens, imagine separar uma frase em palavras

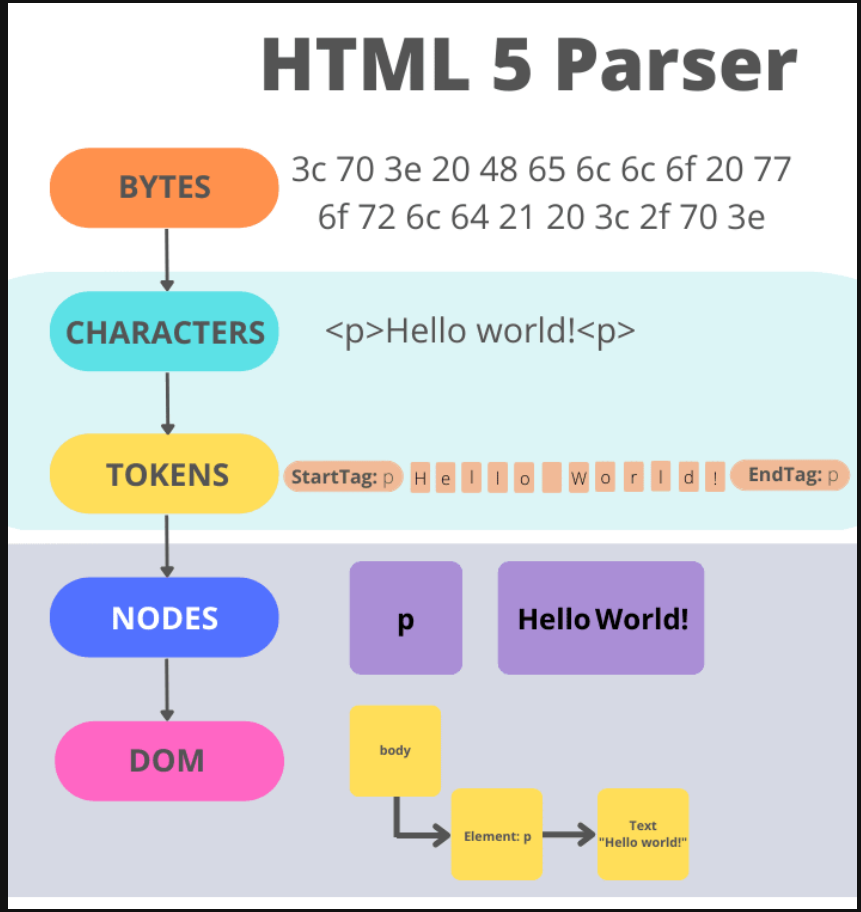


DOM

Depois que o primeiro token é criado, começa o processo de *tree building*, que cria uma estrutura chamada DOM(*Document Object Model*), e vai adicionando os tokens gerados.

A árvore é montada começando pelo elemento root `<html>` e vai refletindo a arquitetura utilizada entre as tags, definindo `parent nodes` e `child nodes`, quanto mais *nodes*, mais tempo vai levar para criar o DOM tree.

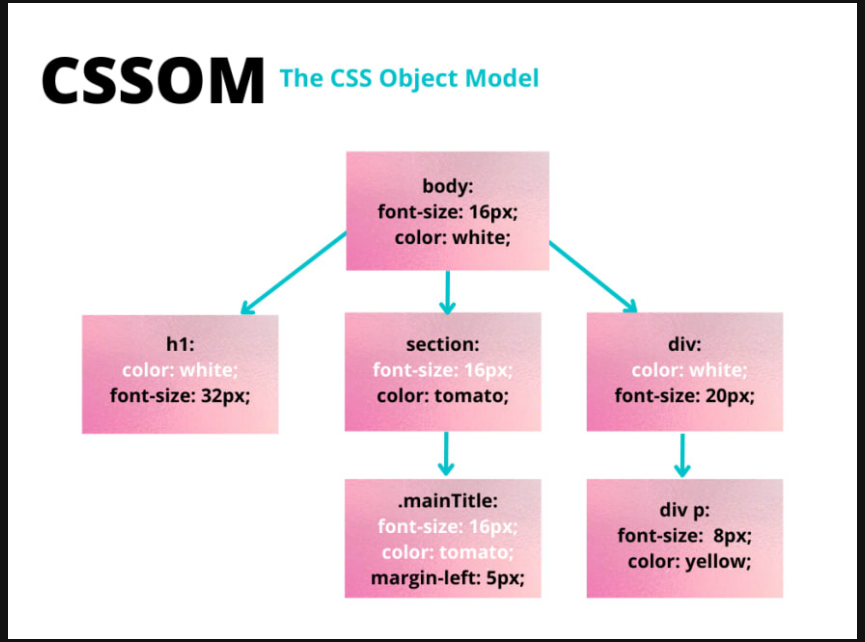
Enquanto o DOM é criado conforme novos tokens são gerados, eles são adicionados ao DOM. Ao encontrar recursos como CSS stylesheets, Javascript files, fontes de texto ou bibliotecas, o parser para a execução, espera o download desses arquivos para continuar a geração.



Parsing CSS

```
body {
  font-size: 16px;
  color: white;
}
h1 {
  font-size: 32px;
}
section {
  color: tomato;
}
section .mainTitle {
  margin-left: 5px
}
div {
  font-size: 20px;
}
```

Se existir folha de estilos, é realizado o *parsing* do CSS sendo criado o CSSOM(*CSS Object Model*). Utilizando Tokenization, o parser recebe os bytes, converte em caracteres, depois tokens, depois nodes e depois eles são conectados no CSSOM.



Javascript

Quando existe Javascript no response, ele deve ser primeiro compilado e/ou interpretado, para isso é utilizado um motor Javascript. Dentre os motores mais utilizados estão, V8, Javascript core, Chakra e SpiderMonkey.

Compilação

No processo de compilação, o código escrito em uma linguagem de alto nível é convertido em linguagem de máquina gerando um object file, para depois ser executado ou não.

Interpretação

Durante o processo de interpretação o código é "lido" linha a linha e executado imediatamente.

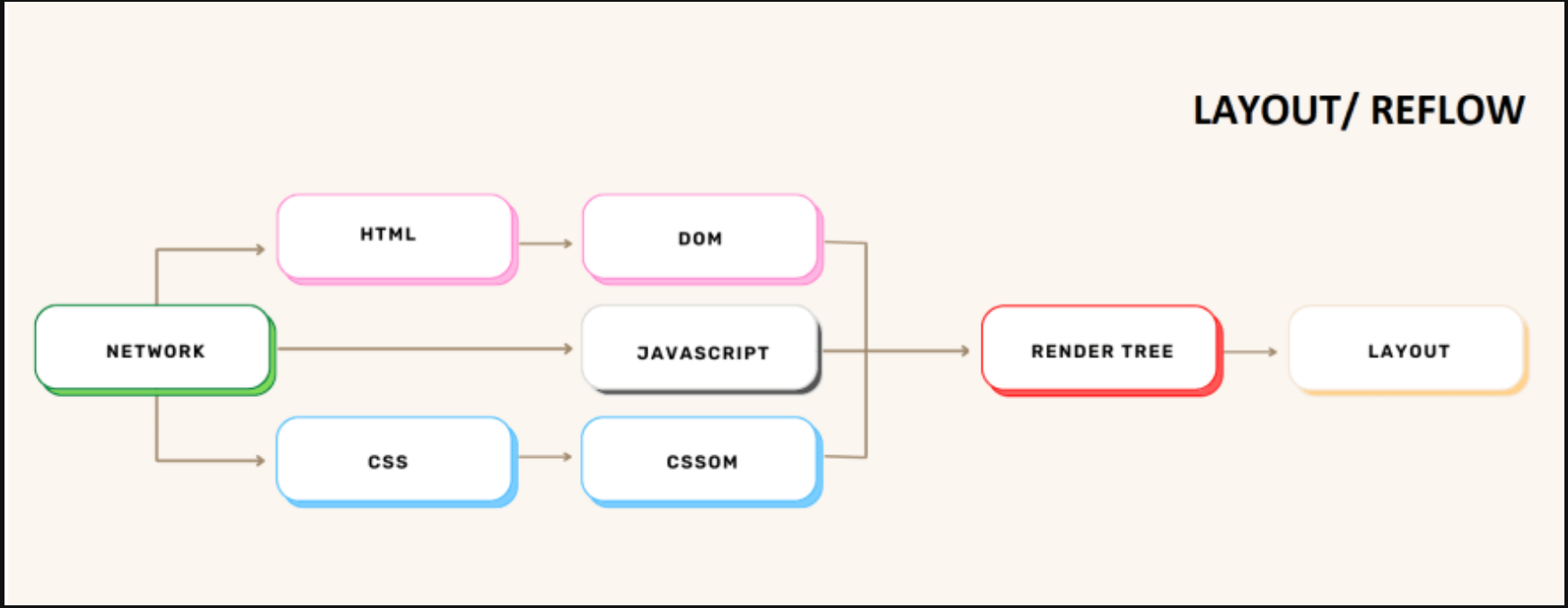
JIT

JIT(*Just in time compilation*) e trabalha intercalando compilação e interpretação. Dessa forma o código é traduzido para linguagem de máquina enquanto é executado. Essa execução vai compilar o código para linguagem de máquina.

Apesar de Javascript ser uma linguagem interpretada e não precisar de compilação, a maioria dos navegadores utiliza JIT

Paint

Após todo o processo visto anteriormente o navegador processa todo o conteúdo gerado e organiza o layout para ser renderizado.



Referências

https://www.w3.org/People/Berners-Lee/

https://en.wikipedia.org/wiki/Uniform_Resource_Identifier#Conception

https://medium.com/@rohitpatil97/http-request-http-response-context-and-headers-part-iii-5c37bd4cb06b

https://www.iana.org/assignments/uri-schemes/uri-schemes.xhtml

https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

https://www.rfc-editor.org/rfc/rfc9110.html

https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work#overview

https://dev.to/arikaturika/how-web-browsers-work-part-1-with-illustrations-1nid

https://www.w3.org/Style/CSS20/history.html

https://developer.mozilla.org/en-US/docs/Glossary/Call_stack

https://dev.to/arikaturika/how-web-browsers-work-the-render-tree-part-7-with-illustrations-24h3